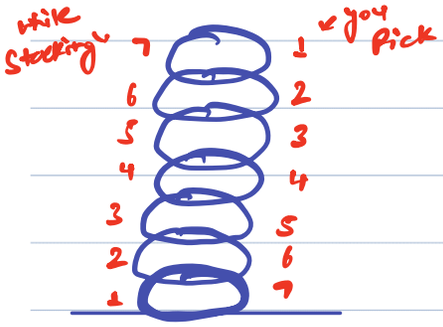
Today's agenda

- Stacks
- Linkedlist as stack
- Remove adjacent duplicate
- Balanced Parentheses
- Min stack

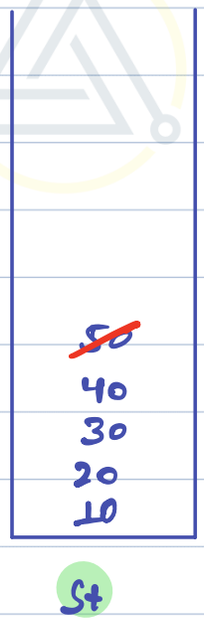
AlgoPrep

//Stacks → Last in first out.



→ Stack <Integer> St = new Stack<>();

(type: Integer; name: St)



Operations:

O(1) ← St.push(10); → add in stack.

O(1) ← St.pop(); → remove topmost no. and return.

O(1) ← St.peek(); → returns you the topmost element.

O(1) ← St.size(); → no. of elements in stack.

Ex: (i) Pile of Plates
(ii) bangles in hand
(iii) undo/redo

Array
Hashmap
Stack

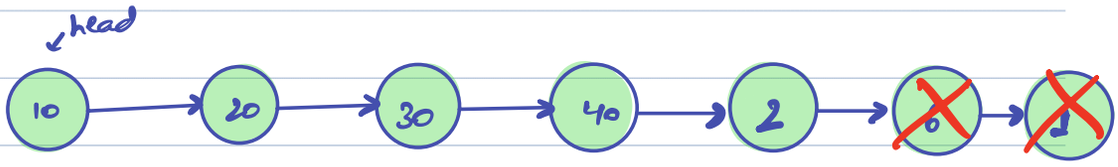
Linkedlist
Tree

AlgoPrep

→ adapters

//Linkedlist as Stack → Last in first out

LL:



2 6 1 POP() POP()

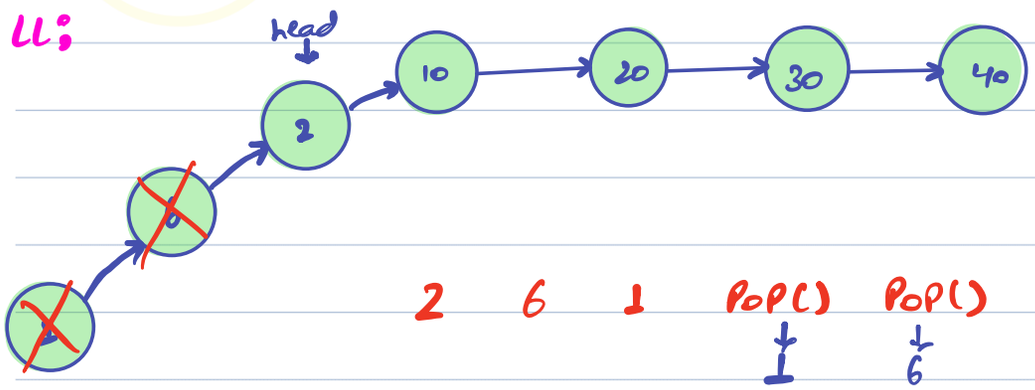
↓ 1 ↓ 6

(i) addLast → O(n)

(ii) removeLast → O(n)

AlgoPrep

LL:



2 6 1 POP() POP()

↓ 1 ↓ 6

(i) add first → O(1)

(ii) remove first → O(1)

Q) Remove adjacent duplicate

↳ Given a string S, Remove equal pair of adjacent characters. Return the final string.

Ex1: a ~~b~~ ~~b~~ ~~b~~ ~~b~~ d → ad

Ex2: a ~~x~~ ~~x~~ ~~x~~ ~~x~~ d e → ade

Ex3: a ~~x~~ ~~x~~ b e → abe

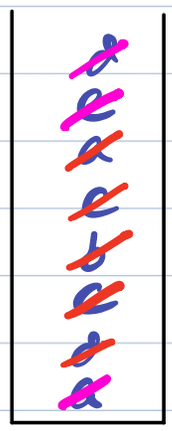
Ex4: a d c ~~x~~ ~~x~~ e ~~x~~ ~~x~~ ~~x~~ ~~x~~ d e d
↳ adceded

Ex5: a ~~x~~ ~~x~~ ~~x~~ ~~x~~ d a
↳ ada

Ex:

0	1	2	3	4	5	6	7	8	9	10	11	12
a	d	c	b	b	c	c	a	a	c	d	e	d

i



dea
↳ aed

//Pseudo code

T.C: O(N)

S.C: O(N)

```
String RemoveAdjacentElement (String s){

    Stack<Character> st = new Stack<>();

    for (int i=0; i< s.length(); i++ ){

        if (st.size()==0 || st.Peek() != s.charAt(i)){
            st.push(s.charAt(i));
        }
        else {
            st.pop();
        }

    }

    Char[] arr = new char[st.size()];
    for (int i= arr.length-1; i>=0; i--){
        arr[i] = st.pop();
    }

    -> char arr to String.

}
```

Break till 9:15Pm

Q) Valid Parentheses

↳ Given a String with '(' ')' '{' '}' '[' ']', you have to find whether the String is balanced or not.

Note: balanced Strings:

(i) open brackets must be closed by the same type of bracket.

S: ( ) { } [ ] (( )) → true

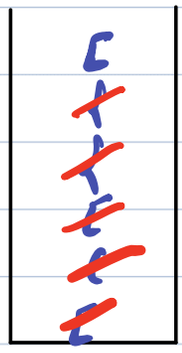
S: ( ) { } [ ) → false

(ii) opening brackets must be closed in correct order.

S: [{] } → false

S: ( ) { (}) → false

	0	1	2	3	4	5	6	7	8	9	10
S:	[	(	)	[	]	{	}	]	{	}	[



//Psuedo code

```
boolean validParentheses (String s){
        Stack<Character> st = new Stack<>();

        for(int i=0; i<s.length(); i++){
            if(st.size()==0 || s[i]=='(' || s[i]=='[' || s[i]=='{'){
                st.push(s[i]);
            }
            else {
                if (s[i]==')'){
                    if (st.peek()=='(') {st.pop();}
                    else {return false;}
                }
                else if (s[i]==']'){
                    if (st.peek()=='[') {st.pop();}
                    else {return false;}
                }
                else {
                    if (st.peek()=='{') {st.pop();}
                    else {return false;}
                }
            }
        }

        if (st.size()==0){return true;}
        else {return false;}
}
```

T.C: O(n)

S.C: O(n)

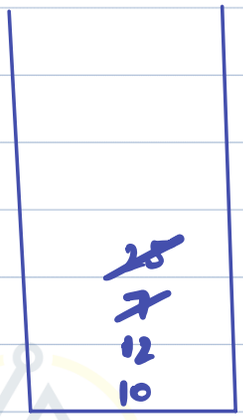
Q) Min Stack

↳ Normal Stack → Pop(), Push(val), Peek(), size()

+

getMin() → min element of Stack

↳ expected T.C: O(1)

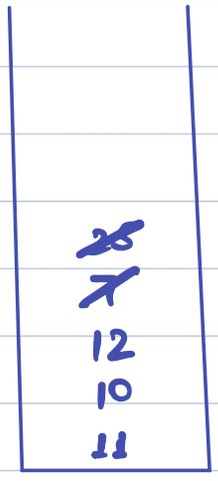


10 12 7 25 getmin() → 7 PoP() → 25

PoP() → 7 getmin() → 10

wrong

// idea!



11 10 12 7 25 getmin() → 7

PoP() → 25 PoP() → 7 getmin()

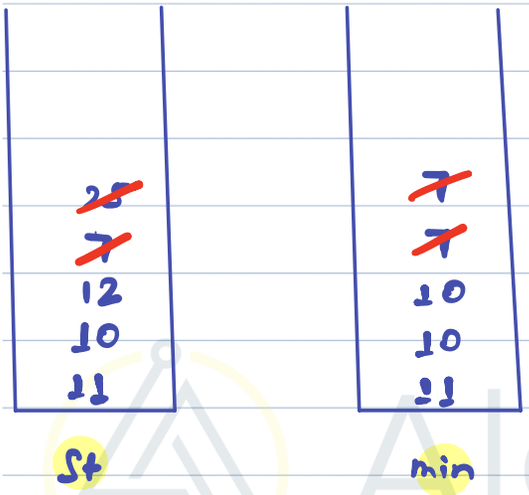
min = ~~10~~ ~~11~~ ~~10~~ ~~7~~ 10

2ndmin = 10

AlgoPrep

//Correct idea 1

↳ use 2 Stacks.



11 10 12 7 25 getmin()
↓
7

PoP() PoP() getmin()
↓ ↓ ↓
25 7 10

T.C: O(N) S.C: O(N)

AlgoPrep